

Tecnologías de Computación de Datos Masivos

Presentación de la asignatura

Curso 2026/2027

Diego Sevilla

Departamento de Ingeniería y Tecnología de Computadores
Facultad de Informática · Universidad de Murcia

dsevilla@um.es



Tecnologías de Computación de Datos Masivos

Curso académico	2026/2027
Titulación	Máster en Big Data
Cuatrimestre	Primero
Curso	Primero
Carácter	Obligatoria
Créditos ECTS	4,5
Departamento	Ingeniería y Tecnología de Computadores

Profesor y horario

Actividad	Profesor	Horario	Aula
Teoría	D. Sevilla	Jueves, 17:30–19:00	Lab. 2.4
Sesiones	D. Sevilla	Jueves, 19:00–20:30	Lab. 2.4

Tutorías y contacto

Profesor	Tutorías	Contacto
D. Sevilla	Miércoles, 11:00–14:00 y tutorías electrónicas	dsevilla@um.es 868 88 7571 Despacho 3.31

Sesiones 1-4 · Fundamentos y motores

- **S1 · Entorno Hadoop con contenedores**
 - Big Data, modelo MapReduce y procesamiento distribuido
 - Clúster local, HDFS, YARN, NameNode, DataNodes y NodeManagers
- **S2 · TPC-DS en Parquet sobre HDFS**
 - TPC-DS SF1 combinado de forma determinista con municipios del INE
 - Rutas, esquemas y lectura mediante WebHDFS, PyArrow, Polars y DuckDB
- **S3 · Data lake sobre almacenamiento S3-compatible**
 - Objetos, buckets y prefijos frente a ficheros y bloques HDFS
 - RustFS, API S3 y lectura de los mismos Parquet con distintas herramientas
- **S4 · Spark como motor distribuido**
 - DataFrames, transformaciones, acciones, joins, ventanas y planes
 - Ejecución local y sobre YARN, particiones, driver y ejecutores

Sesiones 5–8 · Del data lake al lakehouse

- **S5 · Arquitectura medallion y particionado físico**
 - Capas *raw*, *silver* y *gold*, contratos de datos y validación
 - Particiones por fecha, poda de particiones y comparación de planes
- **S6 · Hive Metastore, tablas Hive y Trino**
 - Tablas externas y administradas, registro y descubrimiento de particiones
 - Un metastore compartido por Spark y Trino, con estadísticas `ANALYZE`
- **S7 · Iceberg, evolución y optimización**
 - Snapshots, evolución de esquema, consultas históricas y particionado
 - Compactación, ordenación física, estadísticas y Z-order
- **S8 · Ingesta continua con Structured Streaming**
 - Micro-lotes simulados, checkpoints y `foreachBatch`
 - `MERGE INTO` incremental en *silver* y *gold* con tablas Iceberg

Planificación del curso

Fecha	Sesión
01/10	Introducción a la asignatura
08/10	S1 · Entorno Hadoop con contenedores
22/10	S2 · Exploración de TPC-DS en Parquet sobre HDFS
29/10	S3 · Un data lake sobre almacenamiento S3-compatible
05/11	S4 · Spark como motor de procesamiento distribuido
12/11	S5 · Buenas prácticas de data lake y particionado físico
18/11 y 19/11	S6 · Hive Metastore, tablas Hive y Trino
26/11	S7 · Iceberg: del data lake al lakehouse, evolución y optimización
03/12	S8 · Ingesta continua con Spark Structured Streaming

Calendario de las sesiones

OCTUBRE 2026

Lu	Ma	Mi	Ju	Vi	Sá	Do
			1	2	3	4
5	6	7	8	9	10	11
12	13	14	15	16	17	18
19	20	21	22	23	24	25
26	27	28	29	30	31	

NOVIEMBRE 2026

Lu	Ma	Mi	Ju	Vi	Sá	Do
						1
2	3	4	5	6	7	8
9	10	11	12	13	14	15
16	17	18	19	20	21	22
23	24	25	26	27	28	29
30						

DICIEMBRE 2026

Lu	Ma	Mi	Ju	Vi	Sá	Do
	1	2	3	4	5	6
7	8	9	10	11	12	13
14	15	16	17	18	19	20
21	22	23	24	25	26	27
28	29	30	31			

CONTENIDO DE LA SEMANA

01/10 Introducción a la asignatura
08/10 S1 · Entorno Hadoop
12/10 (sin clase)
22/10 S2 · Parquet TPC-DS en HDFS
29/10 S3 · Data lake S3-compatible

05/11 S4 · Spark y YARN
12/11 S5 · Medallion y particionado
18-19/11 S6 · Hive Metastore y Trino
26/11 S7 · Iceberg y optimización

03/12 S8 · Streaming con Spark

Un mismo recorrido de datos durante las sesiones

Las sesiones trabajan sobre un dataset común y reproducible:

```
TPC-DS SF1                                Relación de municipios del INE
(ventas, clientes, fechas)                (municipio y provincia españoles)
      \                                     /
      \   cruce determinista   /
      ↓
  /datalake/raw/tpcds en HDFS
  Parquet con ciudades y provincias españolas
      ↓
S3 -> Spark/YARN -> silver/gold -> Hive/Trino -> Iceberg/streaming
```

S2 realiza la descarga, normalización y materialización; las sesiones posteriores consumen el estado que deja el notebook anterior. El CI ejecuta esos mismos notebooks y comprueba sus salidas, mientras que el arranque de contenedores queda como preparación del entorno

Evaluación

Teoría · 50 %

- Examen teórico tipo test

Sesiones · 50 %

- Documentación de las sesiones
- Entrevista final

Hay que superar cada parte por separado

Cómo se revisan las sesiones prácticas

Cada guion de sesión termina con una sección **«Evidencias para la siguiente sesión»** que indica qué preparar

En una sesión posterior, cada estudiante tiene una **reunión individual breve** (unos minutos) con el profesor para revisarlas:

- **En el ordenador:** el entorno funcionando y las evidencias pedidas (clúster arrancado, órdenes ejecutadas, interfaces web)
- **Por escrito:** una memoria breve (una o dos páginas) que explique lo realizado, no sólo capturas de pantalla

Este esquema se repite en todas las sesiones prácticas del curso

Bibliografía recomendada

Referencias principales

- Tom White, *Hadoop: The Definitive Guide*, 4.^a ed., O'Reilly, 2015
- Bill Chambers y Matei Zaharia, *Spark: The Definitive Guide*, O'Reilly, 2018
- Holden Karau, Andy Konwinski, Patrick Wendell y Matei Zaharia, *Learning Spark*, O'Reilly, 2015
- Fabian Hueske y Vasiliki Kalavri, *Stream Processing with Apache Flink*, O'Reilly, 2019

Otras referencias

- Petar Zečević y Marko Bonaći, *Spark in Action*, Manning, 2017
- Holden Karau y Rachel Warren, *High Performance Spark*, O'Reilly, 2017
- Sandy Ryza, Uri Laserson, Sean Owen y Josh Wills, *Advanced Analytics with Spark*, O'Reilly, 2017

Repositorio de la asignatura

- Guiones de sesiones, materiales e información adicional
- Repositorio público: <https://github.com/dsevilla/tcdm-public>
- Rama del curso: **26-27**

```
$ git clone https://github.com/dsevilla/tcdm-public.git  
$ cd tcdm-public  
$ git switch 26-27
```

En este caso, como los notebooks requieren el clúster funcionando, no se pueden ejecutar en Colab. Se pueden ejecutar en Visual Studio Code, por ejemplo, conectando con el servidor Jupyter de dentro del clúster